DBMS — Last Day Revision Notes

CSE3001 Database Management Systems | High-Priority Topics + Solved Numericals

Priority Topics Covered

1. DBMS Architecture & Three-Level Schema Architecture
2. ER Diagram (components, symbols, relationships, steps, mapping to relations)
3. Codd's 12 Rules
4. Relational Algebra (operators + solved queries)
5. Keys (Super, Candidate, Primary, Alternate, Foreign)
6. Functional Dependency (types, Armstrong's rules, closure, candidate keys)
7. Normalization — 1NF, 2NF, 3NF (★most important), BCNF, 4NF
8. Role / Functions of DBA
9. Numericals: ER Diagram, Functional Dependency, Relational Algebra

1. DBMS Architecture

The DBMS architecture defines how the components of a database system are organised and how they interact with users and applications. In the commonly taught **three-tier architecture**, the system is split into a Presentation Tier (handles the user interface and the view level), an Application Tier (contains the application server, business logic and abstraction), and a Database Tier (handles query processing, the relation of data and integrity constraints). This separation keeps the user interface, business logic and stored data independent of one another so that each layer can be changed with minimal impact on the others.

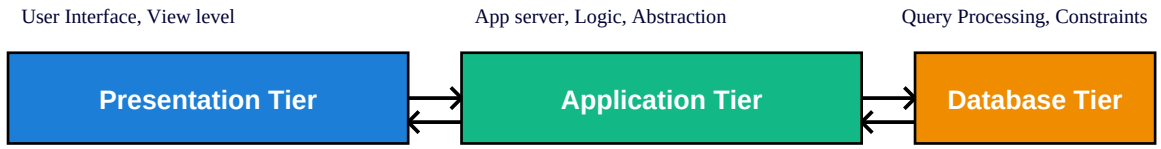


Fig 1.1 — Three-Tier Architecture of DBMS

Internal Structure of DBMS (query flow):

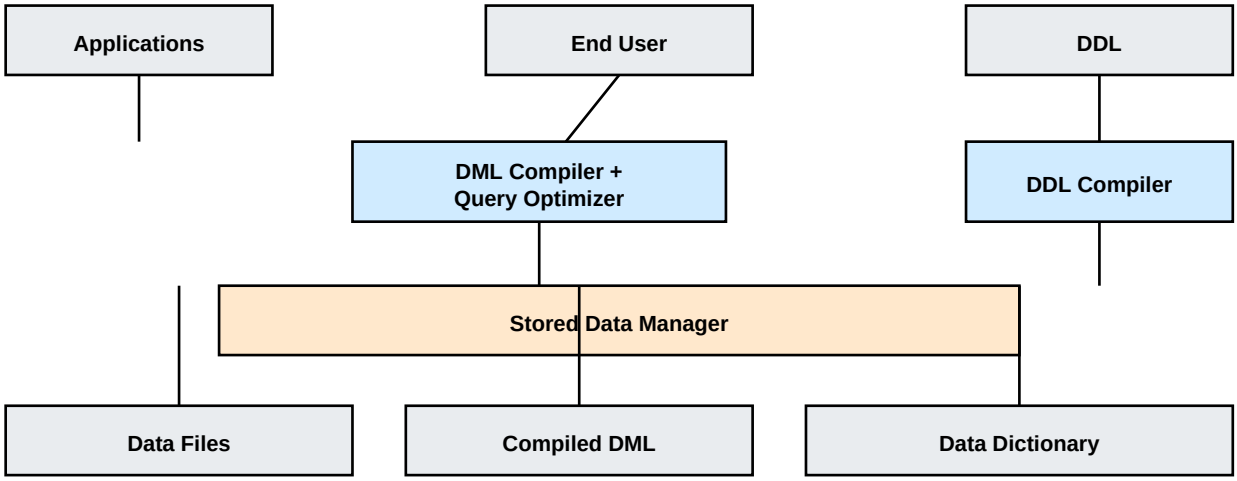


Fig 1.2 — Structure of DBMS (Applications/End-User/DDL flow through compilers to Stored Data Manager)

- DBA (Database Administrator) manages the whole DBMS — schema, security, storage & recovery.
- Data Dictionary stores metadata — table names, columns, indexes, constraints.
- DML Compiler + Query Optimizer converts user queries into an efficient execution plan.
- Stored Data Manager controls access to data stored on disk (data files).

2. Three-Level (ANSI-SPARC) Schema Architecture

The three-level schema architecture separates a database into three levels of abstraction so that each user group sees only the data relevant to it, while physical storage details remain completely hidden. The **Internal level** (physical schema) describes how data is actually stored on disk using a physical data model. The **Conceptual level** (logical level) describes what data is stored and what relationships exist among the data, hiding implementation detail — this is the level at which programmers and DBAs usually work. The **External level** (view level) describes the different views seen by different user groups, hiding the rest of the database from them. This layering gives rise to **data independence** — the ability to change one level without affecting

the levels above it.

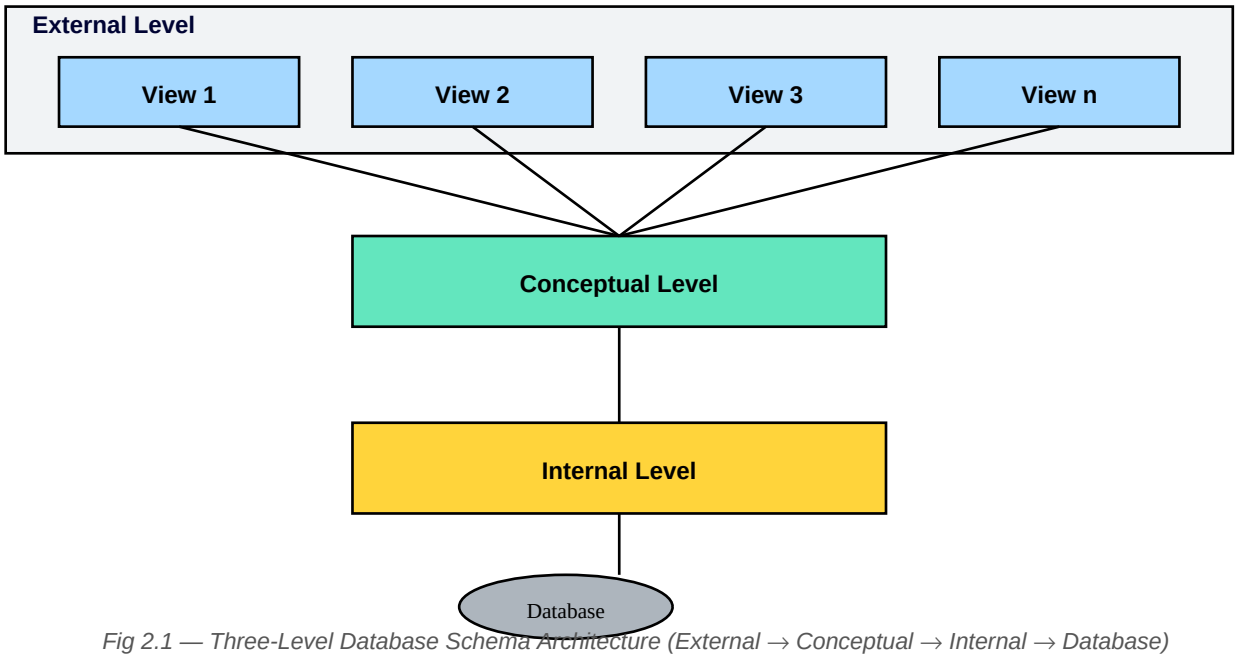


Fig 2.1 — Three-Level Database Schema Architecture (External → Conceptual → Internal → Database)

Data Independence:

- **Physical Data Independence:** ability to change the internal/physical schema without affecting the conceptual schema.
- **Logical Data Independence:** ability to change the conceptual schema without affecting external views/applications.

Database Languages (quick recap):

- **DDL** — Create, Alter, Drop, Truncate, Rename (defines structure).
- **DML** — Select, Insert, Update, Delete (manipulates data).
- **DCL** — Grant, Revoke (access control).
- **TCL** — Commit, Rollback (transaction control).

3. ER (Entity-Relationship) Diagram

An **ER Model** is a high-level conceptual data model diagram used to analyse data requirements and produce a well-designed database. It gives a graphical representation of real-world entities and the relationships between them, and helps explain the logical structure of a database before it is converted into tables.

Components of ER Model:

- **Entity** — an object with physical existence (person, car) or conceptual existence (job, course); represented as a rectangle. A **weak entity** (double rectangle) has no sufficient attributes to form its own primary key.
- **Attribute** — describes a property of an entity; represented as an oval. Types: Key (underlined), Simple/Composite, Single-valued/Multi-valued (double oval), Stored/Derived (dashed oval).
- **Relationship** — association among entities; represented as a diamond.

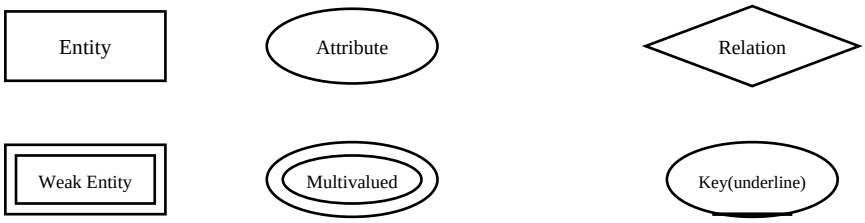


Fig 3.1 — Common ER symbols (Entity, Weak Entity, Attribute, Multivalued Attribute, Key Attribute, Relationship)

Types of Relationships:

- **One-to-One (1:1)** — e.g. a Person *has* one Passport.
- **One-to-Many (1:M)** — e.g. a Customer *places* many Orders.
- **Many-to-One (M:1)** — e.g. many Students *study* in one College.
- **Many-to-Many (M:N)** — e.g. Students *assigned* to many Projects and vice-versa.

Participation Constraints:

- **Total participation** — every entity in the set must occur in the relationship (shown with a double line). E.g. every Department must have a Manager.
- **Partial participation** — an entity may or may not participate in the relationship (single line). E.g. not every Employee manages a Department.

Extended Features:

- **Generalization** — bottom-up process combining lower level entities (e.g. Employee, Customer) into a higher level generalized entity (Person).
- **Specialization** — top-down process splitting an entity (Account) into sub-entities (Saving A/c, Current A/c) with distinct attributes; shown with an “ISA” triangle.
- **Aggregation** — treats a relationship set as a higher-level entity so it can participate in another relationship (a relationship of a relationship).

Steps to Draw an ER Diagram:

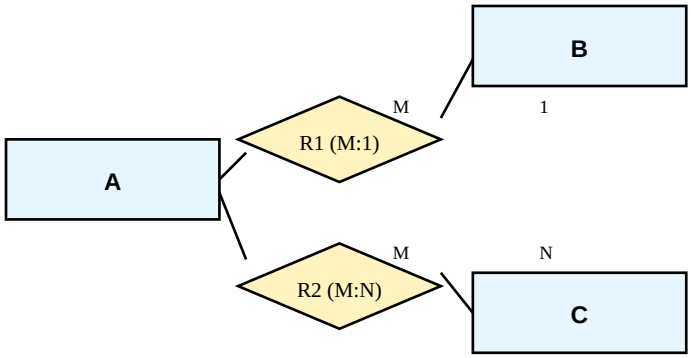
- Step 1 — Entity Identification (find nouns: Student, Course, Professor).
- Step 2 — Relationship Identification (find verbs: assigned, delivers).
- Step 3 — Cardinality Identification (1:1, 1:M, M:1, M:N based on the problem statement).
- Step 4 — Identify Attributes and attach each to exactly one entity.
- Step 5 — Draw the complete ERD combining entities, relationships, cardinalities and attributes.

Rules to convert ER Model into Relational Model (tables):

- Every entity becomes a table; its key attribute becomes the primary key.
- A composite attribute is flattened into its component columns; a multivalued attribute becomes a separate table (with FK to the owner entity).
- **1:M relationship** → the primary key of the “one” side is added as a foreign key on the “many” side (2 tables total; no separate relation table needed).
- **M:N relationship** → a new (3rd) relation is created with the primary keys of both entities as a composite key/foreign keys.
- **1:1 relationship** → primary key of either entity may be posted as a foreign key into the other (2 tables total).

Solved Numerical — ER Diagram → Minimum Tables

Question: Given the ER diagram below, find the minimum number of tables required when it is converted into a relational model.



Entity A relates to B via R1 (Many:1) and to C via R2 (Many:Many)

Solution:

- R1 is Many:1 (A to B) → no extra table; post B's key as a foreign key into A's table.
- R2 is Many:Many (A to C) → a separate relation is required to store R2, with keys of A and C.
- So minimum tables required = Table(A) + Table(B) + Table(C) + Table(R2) = **4 tables**.

General rule used in such questions (from the module's worked examples):

- 1:1 relationship → minimum tables = 2 (no separate relation table needed).
- 1:M / M:1 relationship → minimum tables = 2 (FK posted on the 'many' side).
- M:N relationship → minimum tables = 3 (separate relation table with both keys as composite/foreign key).
- For a chain of entities with mixed relationship types, add 1 extra table for every M:N relationship in the chain; 1:1 and 1:M relationships do not need an extra table.

4. Codd's 12 Rules

Codd's Rules were proposed by **Dr. E. F. Codd**, and comprise a set of 13 rules (numbered 0–12) that define what is required for a database management system to be considered truly **relational**. A DBMS is called “fully relational” only if it satisfies all these rules.

Rule 1 – Information Rule	All data (user data and metadata) must be represented as values in table cells.
Rule 2 – Guaranteed Access	Every data value must be accessible via table name + primary key + column name.
Rule 3 – Systematic NULL Treatment	NULL is handled uniformly to represent missing/unknown/not-applicable data.
Rule 4 – Dynamic Online Catalog	The database description (data dictionary) is stored as data and is queryable using the same language.
Rule 5 – Comprehensive Data Sub-language	A single language must support data definition, manipulation and transaction control.
Rule 6 – View Updating	All theoretically updatable views must be updatable by the system.
Rule 7 – High-Level Insert/Update/Delete	Set-level (not just row-at-a-time) insert, update, delete must be supported.
Rule 8 – Physical Data Independence	Changing physical storage must not affect applications.
Rule 9 – Logical Data Independence	Changing the logical/conceptual schema must not affect applications/views.
Rule 10 – Integrity Independence	Integrity constraints must be definable independently of application programs.
Rule 11 – Distribution Independence	Data distribution across sites must be invisible to end users.
Rule 12 – Non-Subversion	Low-level (record-at-a-time) access must not bypass integrity rules enforced at the relational level.

5. Relational Algebra

Relational Algebra is a **procedural query language** — it specifies both what data is needed and how to retrieve it. It operates on relations (tables) and produces a new relation as the result, and consists of six fundamental operators: select, project, union, set-difference, Cartesian product and rename. Additional (derived/extended) operators — intersection, join, division and aggregate functions — are built using these fundamental ones.

Fundamental Operators:

Operator	Symbol / Notation	Meaning
Select	σ■(r)	Selects rows (tuples) of r satisfying predicate p.
Project	Π A1,...,Ak(r)	Selects given columns; duplicate rows removed.
Union	r ∪ s	All tuples in r or s (r, s must be union-compatible).
Set Difference	r – s	Tuples in r but not in s.
Cartesian Product	r × s	Combines every tuple of r with every tuple of s.
Rename	ρ x (r)	Renames a relation (and optionally its attributes).

Extended / Derived Operators:

- **Set Intersection** (r ∩ s): tuples common to both r and s; r ∩ s = r – (r – s).
- **Join (■)**: combination of Cartesian product + selection on a matching condition — e.g. r ■$_{A=C}$ s = σ$_{A=C}$(r × s). Natural join automatically equates common attribute names.
- **Division (÷)**: r ÷ s answers "for all" queries — result contains A-values from r that are associated with *every* tuple in s.
- **Aggregate (■)**: $_{G1,...,Gn}$■$_{F1(A1),...,Fn(An)}$(E) groups tuples on G1..Gn and applies aggregate functions (sum, avg, min, max, count).
- **Generalized Projection**: allows arithmetic expressions inside Π, e.g. Π name, limit-balance(credit_info).

Solved Query Examples — Banking Schema

`customer(cname, cstreet, ccity)` • `account(acc_no, branch, balance)` • `loan(loan_no, branch, amount)` • `depositor(cname, acc_no)` • `borrower(cname, loan_no)`

Q1. Find loan numbers with amount > 1200.

```
Π loan_no ( σ amount > 1200 (loan) )
```

Q2. Names of customers having a loan and an account both.

```
Π cname (borrower) ∩ Π cname (depositor)
```

Q3. Names of customers having a loan, an account, or both.

```
Π cname (borrower) ∪ Π cname (depositor)
```

Q4. Names of customers with a loan but NOT an account.

```
Π cname (borrower) – Π cname (depositor)
```

Q5. Names of customers with a loan at the 'Indore' branch.

```
Π cname ( σ branch='Indore' (borrower ■ loan) )
```

6. Keys in DBMS

A **key** is an attribute or set of attributes that helps uniquely identify a row (tuple) in a relation (table) and helps establish relationships between tables. Keys ensure that every record can be uniquely identified, which is essential for enforcing entity and referential integrity in a database.

Key Type	Definition
Super Key	Any set of one or more attributes that can uniquely identify a tuple; may contain extra (non-essent
Candidate Key	A minimal super key — a super key whose proper subset is NOT a super key.
Primary Key	The candidate key chosen by the designer to uniquely identify tuples; only one per relation.
Alternate / Secondary Key	The remaining candidate keys that were not chosen as the primary key.
Foreign Key	An attribute in a (slave/child) table that refers to the primary key of another (master/parent) table,

Note on Foreign Key:

- Requires two tables — a Master/Parent table and a Slave/Child table.
- FK column and PK column need not have the same name, but must have the same data type.
- The master table must have a primary key; the slave table need not have one.
- A slave table may have multiple foreign keys.

Integrity Constraints:

- **Domain constraint** — value must belong to the attribute's defined domain (data type/range).
- **Key constraint** — no two tuples can have the same value for the primary key.
- **Entity integrity** — primary key attribute(s) cannot be NULL.
- **Referential integrity** — a foreign key value must either match a primary key value in the parent table or be NULL.

7. Functional Dependency (FD)

A **Functional Dependency** is a constraint between two sets of attributes in a relation. If R is a relation with attributes A and B, we say **A → B** (“A determines B”) if, whenever two tuples agree on the value of A, they must also agree on the value of B. Here A is called the **determinant** and B is the **dependent** attribute; FD is the basis for detecting redundancy and driving normalization, and was introduced by E. F. Codd.

Armstrong's Axioms (Rules of FD):

- **Reflexivity:** if $\beta \subseteq \alpha$ then $\alpha \rightarrow \beta$ (e.g. AB → B).
- **Augmentation:** if $\alpha \rightarrow \beta$ then $\gamma\alpha \rightarrow \gamma\beta$.
- **Transitivity:** if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$ then $\alpha \rightarrow \gamma$.
- **Union:** if $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ then $\alpha \rightarrow \beta\gamma$.
- **Decomposition:** if $\alpha \rightarrow \beta\gamma$ then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$.

Types of FD:

- **Trivial FD:** A → B where B is a subset of A (e.g. AB → A).
- **Non-trivial FD:** A → B where B is NOT a subset of A.

Closure of FD (F■):

The closure of an attribute set is the complete set of attributes that can be functionally derived from it using Armstrong's axioms. It is calculated by repeatedly adding attributes reachable via given FDs until no more can be added, and is the key tool used to find **candidate keys** — an attribute set X is a candidate key if X■ = all attributes of R and no proper subset of X has this property.

Solved Numerical — Functional Dependency / Candidate Keys

Q1. Closure Example.

Relation: student_details(Roll_No, Name, Marks, Location); FD1: Roll_No → Name, Marks; FD2: Name → Marks, Location.

- {Roll_No}■ = {Roll_No, Name, Marks, Location}
- {Name}■ = {Name, Marks, Location}
- {Marks}■ = {Marks}
- {Location}■ = {Location}
- Since {Roll_No}■ covers all attributes, **Roll_No is the candidate key.**

Q2. Find candidate keys of R(A,B,C,D,E) with FDs {AB→CD, D→A, CB→DE}.

- Step 1: Attribute with no incoming edge → B (essential attribute, part of every candidate key).
- Step 2: {B}■ = {B} — does not cover R, so try combinations with B:
- {BA}■ = {A,B,C,D,E} ✓ {BC}■ = {B,C,D,E,A} ✓ {BD}■ = {B,D,A,C,E} ✓ {BE}■ = {B,E} ✗
- **Candidate keys = {AB, BC, BD}**

Q3. Highest normal form of R(A,B,C,D,E) with FD = {BC→D, AC→BE, B→E}.

- Candidate Key: (AC)■ = {A,C,B,E,D} → AC is the candidate key.
- Prime attributes = {A, C}; Non-prime = {B, D, E}.
- 1NF: satisfied (assume atomic values).
- 2NF: BC→D — BC is not a proper subset of AC → OK. AC→BE — AC is the whole key → OK. B→E — B is not a subset of AC → OK. So relation is in 2NF.
- 3NF: BC→D — BC is not a super key and D is not prime → **violates 3NF**. B→E — B is not a super key and E is not prime → **violates 3NF**.
- **Conclusion: highest normal form achieved = 2NF.**

Q4. Minimal (canonical) cover of F = {A→B, B→C, A→C}.

- A→C is redundant since it is derivable from A→B and B→C by transitivity.
- **Minimal cover F_c = {A→B, B→C}**

8. Normalization ★ (1NF, 2NF, 3NF most important)

Normalization is the process of organising the attributes and relations of a database to minimise redundancy and eliminate undesirable insertion, update and deletion anomalies. It works by decomposing a larger table into smaller, well-structured tables and linking them through foreign keys, progressing through a series of "normal forms" (1NF, 2NF, 3NF, BCNF, 4NF, 5NF) each of which removes a specific category of redundancy.

Anomalies (why we normalize):

- **Update anomaly** — the same fact stored in multiple rows must be updated everywhere, or data becomes inconsistent.
- **Insertion anomaly** — cannot insert certain data without also having unrelated data (e.g. can't add an employee without a department if that field is NOT NULL).
- **Deletion anomaly** — deleting one fact unintentionally deletes another unrelated fact.

1NF (First Normal Form)

A relation is in 1NF if every attribute contains only single (atomic) values — no composite or multi-valued attributes are allowed, and all values in a column belong to the same domain.

- Fix: split repeating/multi-valued data into separate rows, or move it into a new related table (preferred, avoids NULLs).

2NF (Second Normal Form)

A relation is in 2NF if it is in 1NF and has **no partial dependency** — i.e. every non-prime attribute is fully functionally dependent on the whole of every candidate key, not on just part of a composite key. 2NF issues only arise when the candidate key is composite.

- **Prime attribute** = part of some candidate key; **Non-prime attribute** = not part of any candidate key.
- **Partial dependency** = a non-prime attribute depends on only part of a composite candidate key.
- Fix: remove the partially-dependent attribute into its own table with the part of the key it truly depends on.

3NF (Third Normal Form)

A relation is in 3NF if it is in 2NF and has **no transitive dependency** — i.e. no non-prime attribute depends on another non-prime attribute; every non-prime attribute must depend directly on a candidate key. Formally: for every FD X→Y, either X is a super key, or Y is a prime attribute.

- Transitive dependency: if A→B and B→C hold (B not a key), then A→C is transitive.
- Classic example: EMP_ID→EMP_ZIP and EMP_ZIP→EMP_CITY, EMP_STATE — city/state are transitively dependent on EMP_ID via EMP_ZIP, so move ZIP→City,State into a separate table.

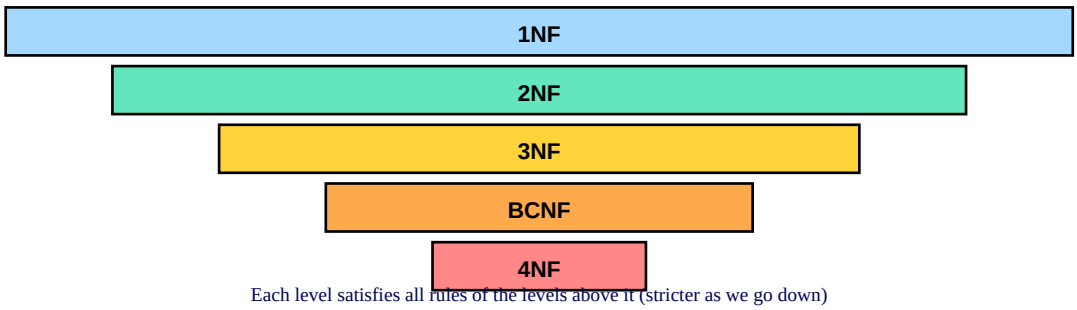


Fig 8.1 — Normal Form hierarchy: 1NF ⊃ 2NF ⊃ 3NF ⊃ BCNF ⊃ 4NF

BCNF (Boyce-Codd Normal Form) — brief

A relation is in BCNF if, for every non-trivial FD X→Y, X must be a super key (stricter than 3NF, which also allows Y to be a prime attribute). BCNF removes redundancy that can survive even in 3NF.

4NF — brief

A relation is in 4NF if it is in BCNF and contains no non-trivial multi-valued dependency ($X \rightarrow\rightarrow Y$ where X is not a super key). Multi-valued dependency occurs when, for a single value of A, several independent values of B and C exist together in the same relation.

Solved Numerical — Normalization up to 3NF

Q. Normalize R(A,B,C,D,E) up to 3NF given FD = {A→BCDE, C→D, B→E}.

- **Check 1NF:** assume all attributes atomic — satisfied.
- **Check 2NF:** (A)■ = {A,B,C,D,E} → Candidate key = {A}. Since key is a single attribute, no partial dependency is possible — relation is already in 2NF.
- **Check 3NF:**
- – A→B: direct, OK. A→C: direct, OK.
- – A→D: transitive via A→C and C→D — **violates 3NF**.
- – A→E: transitive via A→B and B→E — **violates 3NF**.
- **Decompose:** R1(A,B,C) — attributes fully/directly dependent on the key; R2(C,D) — for C→D; R3(B,E) — for B→E.
- Final 3NF relations: **R1(A,B,C), R2(C,D), R3(B,E)**

Quick check procedure to remember for any 3NF question:

- 1. Verify 1NF (atomic values).
- 2. Find candidate key(s) using attribute closure.
- 3. Check 2NF — no non-prime attribute may depend on part of a composite key (skip this check if key is a single attribute).
- 4. Check 3NF — for every FD X→Y, X must be a super key OR Y must be a prime attribute.
- 5. Decompose relations that fail the test, keeping the determinant with its dependents in one relation, and preserving a copy of the determinant in the original relation for joining back (lossless decomposition).

9. Database Administrator (DBA)

A **Database Administrator (DBA)** is the person (or role) responsible for the overall management, control and maintenance of a database system, including its design, security, performance and recovery. The DBA acts as the custodian of both the data and the software that manages it, ensuring the database remains available, consistent and secure for all its users.

Functions of DBA:

- Designing the Conceptual and Physical Schema of the database.
- Security and Authorization — controlling who can access/modify what data.
- Data Availability and Recovery from failures (backup & recovery planning).
- Storage Structure and Access Method definitions (choosing indexes, file organisation).
- Integrity Constraint Specification — defining and enforcing rules that keep data valid.

Last-Minute Formula / Fact Sheet

3-Tier Architecture: Presentation → Application → Database Tier.
3-Level Schema: External (view) → Conceptual (logical) → Internal (physical).
Data Independence: Physical (internal changes don't affect conceptual) & Logical (conceptual changes don't affect external).
Codd's Rules: 12 rules (Information, Guaranteed Access, NULL treatment, Catalog, Sub-language, View Updating, High-level DML, Physical/Logical/Integrity/Distribution Independence, Non-Subversion).
Relational Algebra 6 basic ops: σ select, Π project, ∪ union, – difference, × Cartesian product, ρ rename.
Keys order: Super Key ⊃ Candidate Key → Primary Key (chosen) + Alternate Keys (rest). Foreign Key links tables.
FD candidate key trick: attribute(s) with NO incoming edge are essential and belong to every candidate key; compute closure.
1NF = atomic values only. **2NF** = 1NF + no partial dependency (only matters for composite keys). **3NF** = 2NF + no transitive dependency. **BCNF** = every determinant is a super key.
M:N ER relationship → needs its own table; **1:1 / 1:M** → does not need a separate table (FK suffices).
DBA handles: schema design, security, recovery, storage/access methods, integrity constraints.

Good luck for the exam! Revise the 3NF and Relational Algebra numericals once more before the paper.